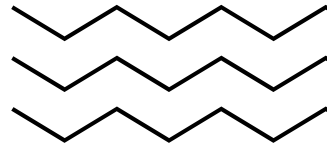


A M I N C H A R E P O O



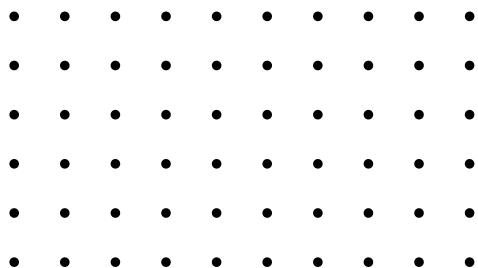
Computer Engineering
Northeastern University

Projects & Technical Work

 charepoo.a@northeastern.edu

 <https://github.com/AminCharepoo>

 <https://www.linkedin.com/in/amin-charepoo/>

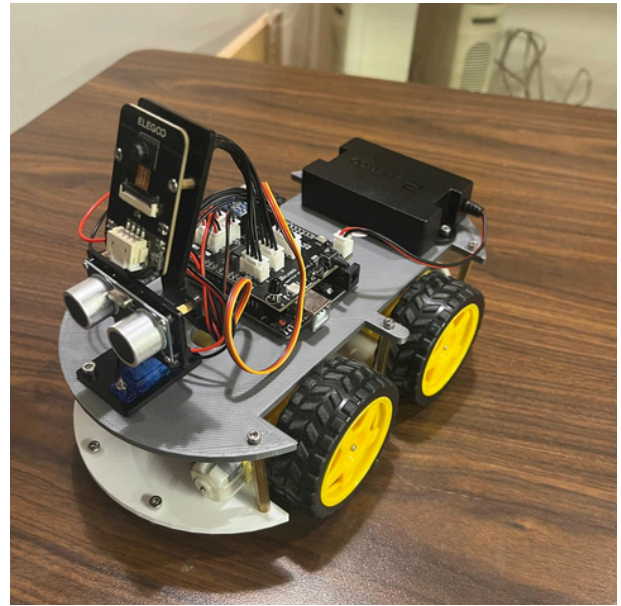


Autonomous Line-Following Robot

<https://github.com/AminCharepoo/Arduino-Line-Following-Robot>

Overview

- Autonomous Arduino robot that follows a line and avoids obstacles using multiple sensors
- Won 1st place in end-of-year competition against 3 other teams
- Built as a hands-on robotics project combining sensing, motor control, and decision-making

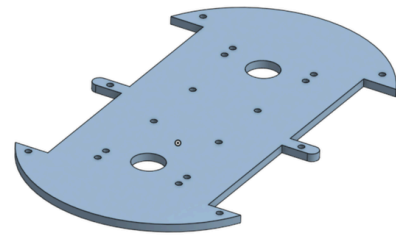
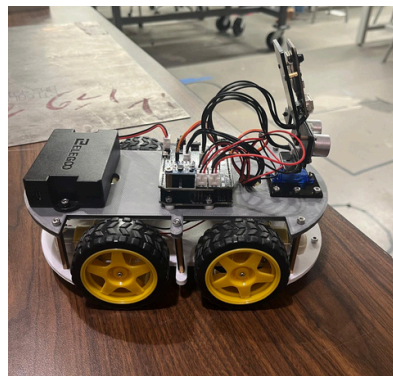


Architecture & Implementation

- Three analog line sensors for left/center/right directional correction
- Swing-turn motor control using PWM
- Ultrasonic sensor on a servo for left/right distance scanning
- Reactive control loop prioritizing line following with obstacle override
- Adjustable thresholds and speed parameters for tuning

Key Challenges & Solutions

- Reduced jerky motion by implementing swing turns instead of sharp pivots
- Improved obstacle decisions using servo-based scanning instead of fixed sensing
- Added recovery behavior when the line is temporarily lost

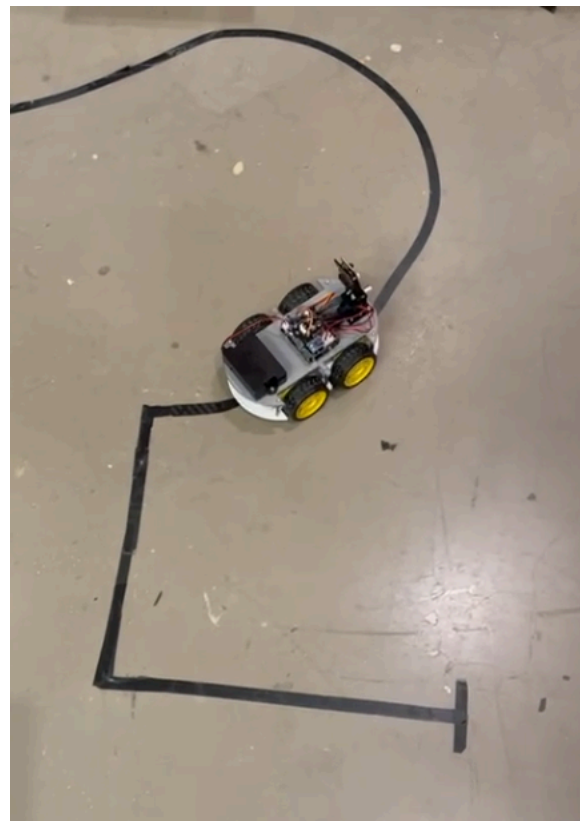


Tech Stack

- Hardware: Arduino Uno, 3x analog line sensors, ultrasonic sensor, servo motor, 4x motors
- Software: Arduino C++
- Tools: Arduino IDE, Git for version control

Outcome

- Fully autonomous navigation with reliable line tracking and obstacle avoidance
- Modular, maintainable code supporting future enhancements
- Deepened understanding of real-time sensor fusion and reactive control systems



Real-Time Task Scheduler

https://github.com/AminCharepoo/Real-Time_Task_Scheduler

Project Overview

- Built for EECE 2140 (Embedded Systems), Spring 2026 with Dani Robinson and Manu Geronimo
- Real-time cooperative task scheduler on Raspberry Pi Pico demonstrating OOP principles
- Manages concurrent tasks (buttons, timers, LCD) with priority-based execution
- Demo: LCD scrolls text; VIP button preempts mid-scroll, displays priority

System Design

- Five classes: Task (abstract base), ButtonTask, TimerTask, DisplayTask, Queue
- Priority-based circular buffer with no dynamic memory allocation
- GPIO and hardware timer ISRs enqueue tasks asynchronously
- Tasks poll for VIP interrupts, save state, yield to priority task, then resume

Key Features

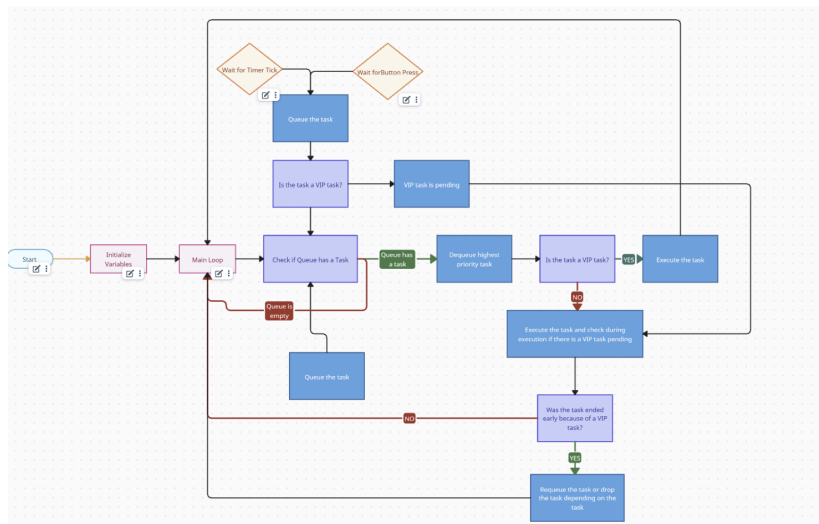
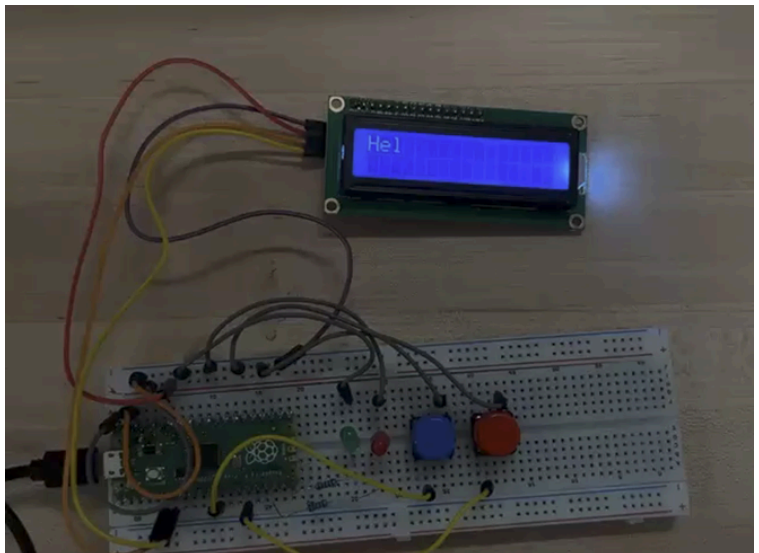
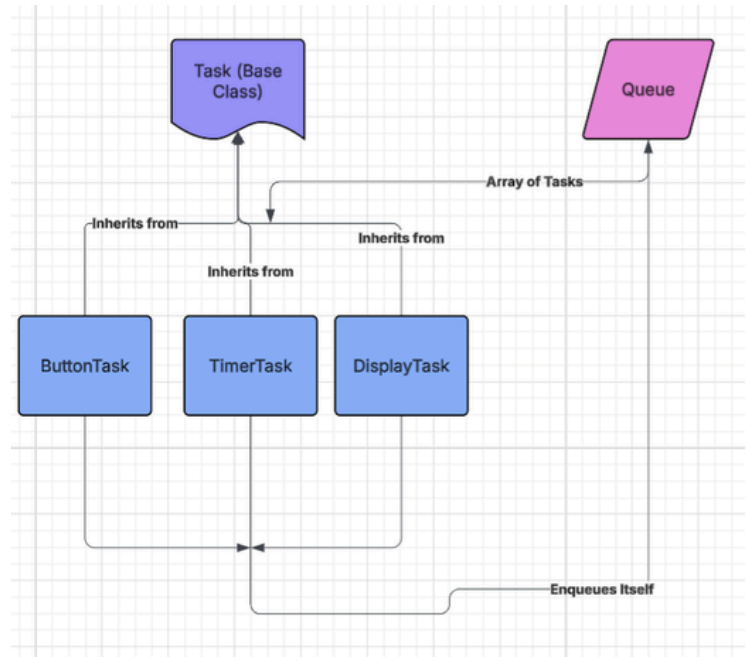
- VIP Preemption: High-priority interrupts preempt tasks cooperatively
- Priority Queue: Fixed-size circular buffer with three priority levels and front-insertion
- ISR-Safe Design: Interrupt handling, and volatile flags
- Cooperative Resumption: Preempted tasks save state and resume after VIP completes

Tech Stack

- Hardware: Raspberry Pi Pico, 16x2 LCD (I2C), GPIO buttons, LEDs
- Software: C++, Pico SDK, VSCode
- Documentation: LaTeX with algorithmic pseudocode and diagrams

Technical Highlights

- OOP: Inheritance, polymorphism, encapsulation
- ISR Patterns: Static pointers and SDK
- LCD Driver: I2C HD44780 with character-by-character scrolling
- No Heap: Fixed arrays ensure deterministic real-time memory usage

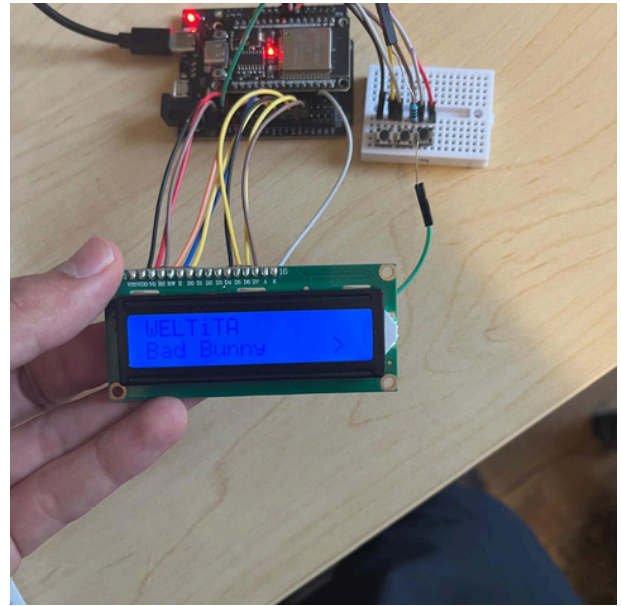


Spotify Controller

<https://github.com/AminCharepoo/SpotifyController>

Overview

- Standalone ESP32-based Spotify controller with physical playback controls and real-time LCD display
- Designed as a hands-free solution to control Spotify without switching apps or unlocking devices
- Custom 3D-printed enclosure with AutoCAD-designed component layout

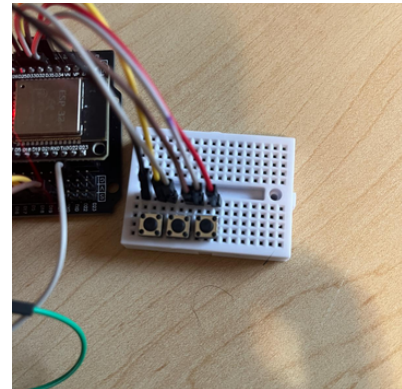


Architecture and Design

- ESP32 connects to WiFi and communicates with Spotify Web API
- REST API requests control playback (play/pause, skip, previous); JSON responses parsed with ArduinoJson
- 16x2 LCD displays current track, artist, and playback state in real-time
- Polling-based update loop refreshes playback info without blocking button input
- Software debouncing ensures reliable button handling

Key Challenges & Solutions

- OAuth on embedded hardware: Implemented automatic token refresh to maintain API access without user re-authentication
- Unreliable button input: Added software debouncing to prevent false triggers
- API choice: Used official Spotify Web API instead of unofficial libraries for long-term reliability

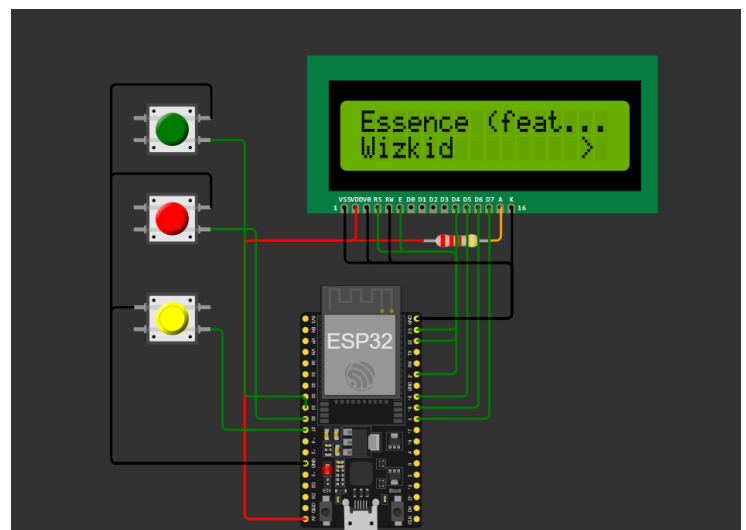


Technical Stack

- Hardware: ESP32, 16x2 LCD (parallel), tactile buttons
- Software: C++, ArduinoJson, Spotify Web API
- Design Tools: AutoCAD, OnShape
- Tools: Arduino IDE, Git

Outcome

- Seamless hands-free Spotify control, eliminating the need to unlock phone or switch apps
- Improved workflow efficiency during focused work sessions
- Gained experience with OAuth flows, REST APIs, and embedded network programming



Wiring Diagram

Gamified Hydration

https://github.com/AminCharepoo/Gamified_Hydration

Overview

- Raspberry Pi Pico hydration tracker that weighs your water bottle and gamifies intake with an animated cat companion
- Cat's mood improves as you hit hydration milestones; audio celebrations reinforce progress
- Built to solve a personal problem: forgetting to drink water despite generic reminder apps

Problem & Motivation

- Generic hydration apps rely on manual logging, which is easy to forget or ignore
- Lack tangible, real-world feedback — just notifications that get dismissed
- Wanted a physical, interactive system with automatic tracking and personalized, engaging feedback

Key Features

- Automatic tracking: HX711 load cell measures water consumption without manual input
- Personalized goals: Daily targets calculated based on body weight and activity level
- Gamified feedback: Animated cat companion with mood states tied to hydration progress
- Audio milestones: Buzzer plays celebration sounds when goals are reached

Technical Highlights

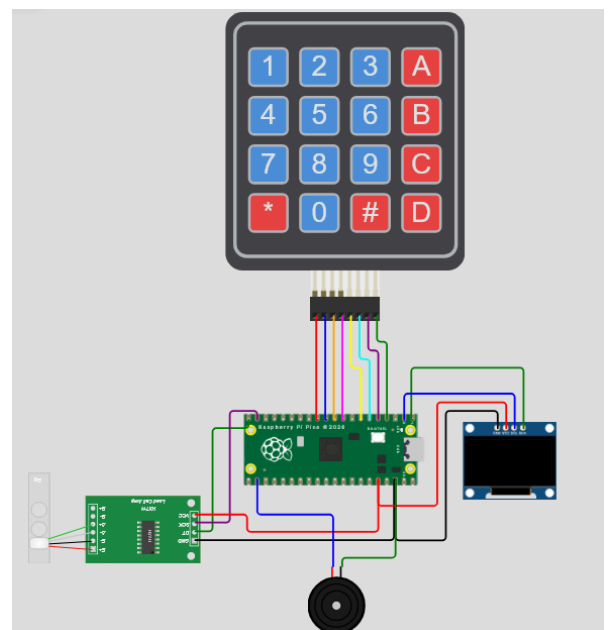
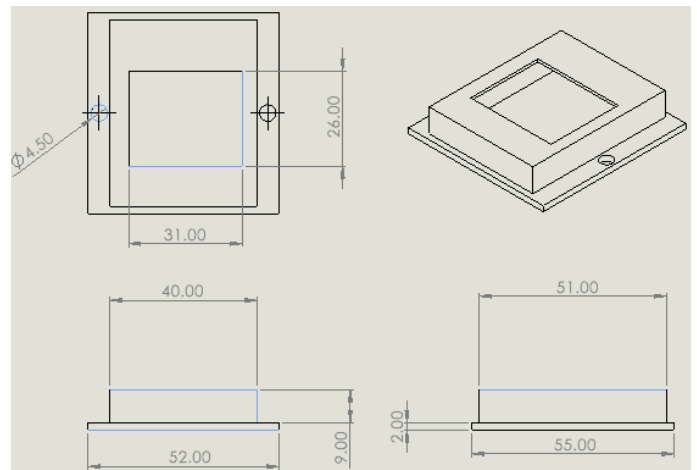
- Embedded state machine managing cat mood transitions based on hydration levels
- Non-blocking timer-based events to avoid blocking UI updates
- Keypad-driven UX design optimized for embedded constraints
- Gamification mechanics designed to reinforce positive behavior without feeling pushy

Tech Stack

- Hardware: Raspberry Pi Pico, HX711 load cell ADC, SSD1306 OLED (I2C), 4x4 matrix keypad, PWM buzzer
- Software: MicroPython

Outcome

- Successfully tracked daily hydration with minimal user input
- Gamification increased drinking consistency
- Gained hands-on experience with sensor calibration, state machines, and embedded UX design



Carnival Game

<https://github.com/AminCharepoo/Maze-Carnival-Game>

Overview

- Automated carnival maze game where players guide a metal ball using a model soldering iron to complete objectives under time pressure
- Built for a multidisciplinary engineering course with a team of three
- Fully self-operating with automated scoring, LED feedback, and audio cues
- Demonstrated at end-of-semester carnival event with 50+ plays



Concept & Design

- Game mechanic based on steady-hand dexterity and time management (faster completion = higher score)
- Themed around short circuits as the conceptual foundation
- Designed maze layout to balance difficulty and approachability for casual players.

My Contributions

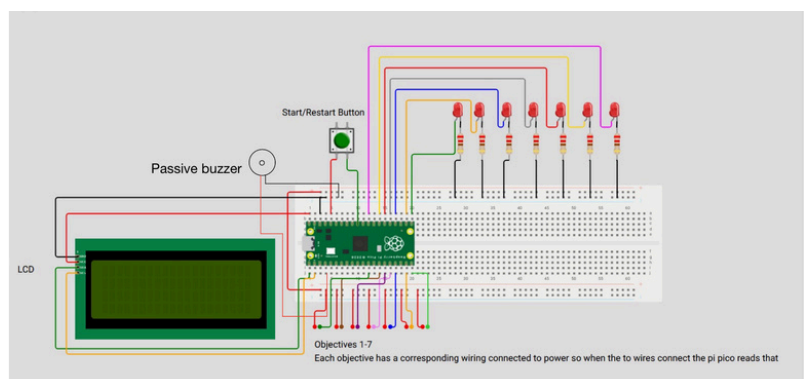
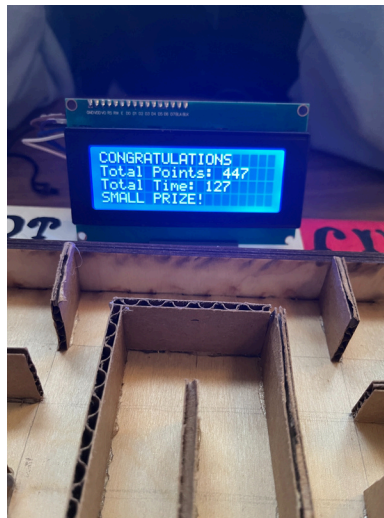
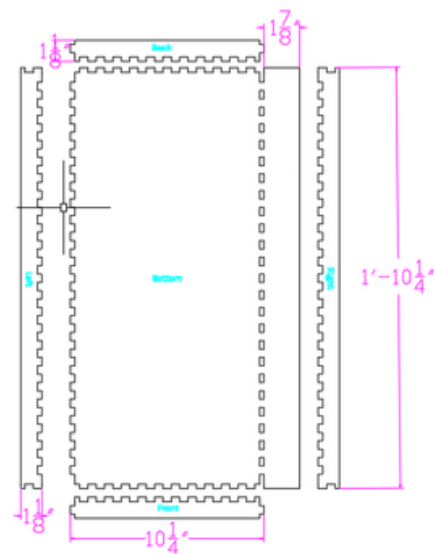
- Software: Programmed ball detection, LED control, buzzer timing, and LCD score display
- Hardware: Wired and integrated Pico with maze sensors for reliable conductivity-based detection
- CAD: Created technical drawings in AutoCAD for laser cutting maze base and walls
- Assembly: Co-designed and constructed physical maze structure

Tech Stack

- Hardware: Raspberry Pi Pico, metal ball maze, LEDs, passive buzzer, 16x2 LCD
- Software: MicroPython
- Tools: Git, AutoCAD

Outcome

- 50+ successful plays with no technical failures
- Gained experience in embedded systems, CAD/fabrication, and hardware/software integration



Stair Lights

<https://github.com/AminCharepoo/Stair-Lights>

Overview

- ESP32-based smart stair lighting system with motion-activated wake, deep sleep power saving, and web-based RGB control
- Built to replace commercial LED systems with a low-power, locally customizable solution
- Supports OTA firmware updates and persistent user settings

Problem & Motivation

- Commercial LED systems consume power continuously even when unused
- Lack customizable local control without proprietary apps or cloud dependencies
- Needed a presence-reactive system with manual override and wireless configuration

Key Features

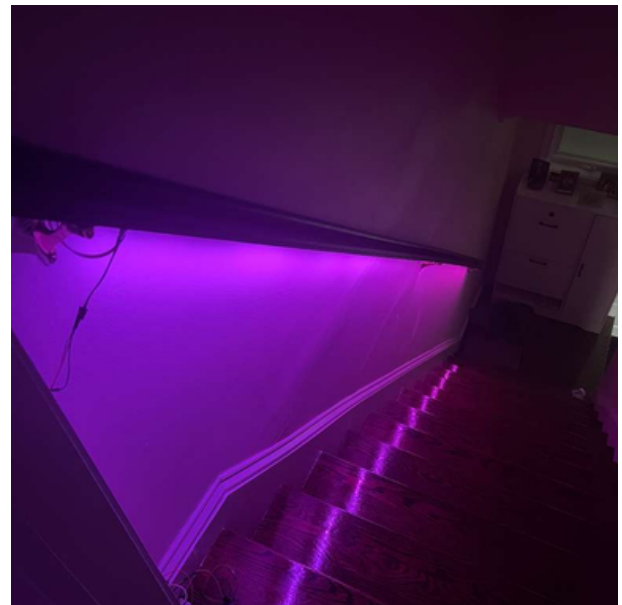
- Low-power operation: Deep sleep mode with GPIO wake-up via PIR sensor or manual button press
- Web-based control: Local interface for RGB color and brightness adjustment (no app required)
- Persistent settings: User preferences stored in ESP32 NVS (non-volatile storage)
- OTA updates: Remote firmware updates via ArduinoOTA without physical access
- Easy setup: Automatic WiFi provisioning using WiFiManager captive portal

Tech Stack

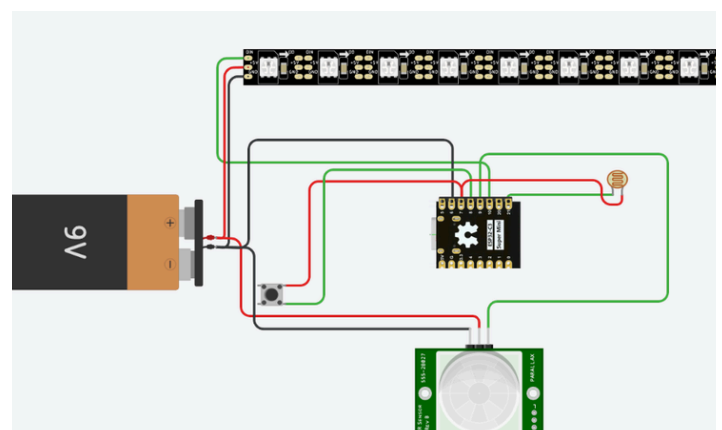
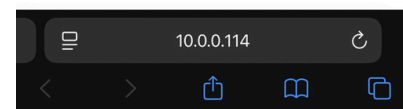
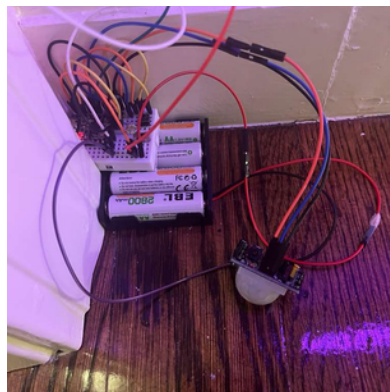
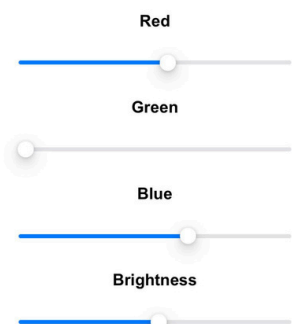
- Hardware: ESP32, LEDs, PIR motion sensor, photoresistor, push button
- Software: Arduino C++, HTML/CSS
- Libraries: Adafruit NeoPixel, WiFiManager, ArduinoOTA

Outcomes

- Successfully deployed on staircase with a week of battery life between charges
- Motion detection reliably triggers lighting
- Web interface enables quick adjustments without reflashing firmware
- Gained hands-on experience with low-power embedded design and wireless IoT systems



ESP32 RGB LED Controller



Wiring Diagram